pandas

Lecture 09

Dr. Colin Rundel

pandas

pandas is an implementation of data frames in Python - it takes much of its inspiration from R and NumPy.

> pandas aims to be the fundamental high-level building block for doing practical, real world data analysis in Python. Additionally, it has the broader goal of becoming the most powerful and flexible open source data analysis / manipulation tool available in any language.

Key features:

- DataFrame and Series (column) object classes
- Reading and writing tabular data
- Data munging (filtering, grouping, summarizing, joining, etc.)
- Data reshaping

```python
import pandas as pd
pd.__version__
```

```
'3.0.0'
```

Series

Series

The columns of a DataFrame are constructed using the `Series` class - these are a 1D array-like object containing values of the same type (similar to a numpy array).

```
pd.Series([1,2,3,4])
```

```
0    1
1    2
2    3
3    4
dtype: int64
```

```
pd.Series(["C","B","A"])
```

```
0    C
1    B
2    A
dtype: str
```

```
pd.Series([True])
```

```
0    True
dtype: bool
```

```
pd.Series(range(5))
```

```
0    0
1    1
2    2
3    3
4    4
dtype: int64
```

```
pd.Series([1,"A",True])
```

```
0       1
1       A
2    True
dtype: object
```

Series methods

Once constructed the components of a series can be accessed via the `array` and `index` attributes.

```
s = pd.Series([4,2,1,3])
```

```
s
```

```
0    4
1    2
2    1
3    3
dtype: int64
```

```
s.array
```

```
<NumpyExtensionArray>
[4, 2, 1, 3]
Length: 4, dtype: int64
```

```
s.index
```

```
RangeIndex(start=0, stop=4, step=1)
```

An index (row names) can also be explicitly provided when constructing a Series,

```
t = pd.Series([4,2,1,3], index=["a","b","c","d"])
```

```
t
```

```
a    4
b    2
c    1
d    3
dtype: int64
```

```
t.array
```

```
<NumpyExtensionArray>
[4, 2, 1, 3]
Length: 4, dtype: int64
```

```
t.index
```

```
Index(['a', 'b', 'c', 'd'], dtype='str')
```

Series + NumPy

Series objects are compatible with NumPy-like functions (i.e. vectorized)

```
t = pd.Series([4,2,1,3], index=["a","b","c","d"])
```

```
t + 1
```

```
a    5
b    3
c    2
d    4
dtype: int64
```

```
np.log(t)
```

```
a    1.386294
b    0.693147
c    0.000000
d    1.098612
dtype: float64
```

```
t / 2 + 1
```

```
a    3.0
b    2.0
c    1.5
d    2.5
dtype: float64
```

```
np.exp(-t**2/2)
```

```
a    0.000335
b    0.135335
c    0.606531
d    0.011109
dtype: float64
```

Series indexing

Series can be subset by their index or by logical expressions (as with NumPy arrays or R vectors)

```
t = pd.Series([4,2,1,3], index=["a","b","c","d"])
```

```
t["c"]
```

```
np.int64(1)
```

```
t[["a","d"]]
```

```
a    4
d    3
dtype: int64
```

```
t[t == 3]
```

```
d    3
dtype: int64
```

```
t[t % 2 == 0]
```

```
a    4
b    2
dtype: int64
```

```
t["d"] = 6; t
```

```
a    4
b    2
c    1
d    6
dtype: int64
```

Position based indexing

Series cannot be directly indexed by position, but the `iloc` *attribute* can be used for this purpose.

```
t[1]
```

```
KeyError: 1
```

```
t[[1,2]]
```

```
KeyError: "None of [Index([1, 2], dtype='int64')] are in the [index]"
```

```
t.iloc[1]
```

```
np.int64(2)
```

```
t.iloc[[1,2]]
```

```
b    2
c    1
dtype: int64
```

Index alignment

When performing operations with multiple series, generally pandas will attempt to align the operation by the index values,

```
m = pd.Series([1,2,3,4], index = ["a","b","c","d"])
n = pd.Series([4,3,2,1], index = ["d","c","b","a"])
o = pd.Series([1,1,1,1,1], index = ["b","d","a","c","e"])
```

```
m + n
```

```
a    2
b    4
c    6
d    8
dtype: int64
```

```
n + m
```

```
a    2
b    4
c    6
d    8
dtype: int64
```

```
n + o
```

```
a    2.0
b    3.0
c    4.0
d    5.0
e    NaN
dtype: float64
```

Series and dicts

Series can also be constructed from dictionaries, in which case the keys are used as the index.

```
d = {"anna": "A+", "bob": "B-", "carol": "C", "dave": "D+"}
pd.Series(d)
```

```
anna     A+
bob      B-
carol     C
dave     D+
dtype: str
```

Index order will follow key order, unless overridden by `index`,

```
pd.Series(d, index = ["dave","carol","bob","anna"])
```

```
dave     D+
carol     C
bob      B-
anna     A+
dtype: str
```

Missing values

Pandas encodes missing values using NaN (mostly),

```
s1 = pd.Series(
  {"anna": "A+", "bob": "B-",
   "carol": "C", "dave": "D+"},
  index = ["erin","dave","carol",
           "bob","anna"]
)
```

```
s2 = pd.Series(
  {"anna": 97, "bob": 82,
   "carol": 75, "dave": 68},
  index = ["erin","dave","carol",
           "bob","anna"],
  dtype = 'int64'
)
```

```
s1
```

```
erin     NaN
dave      D+
carol      C
bob       B-
anna      A+
dtype: str
```

```
s2
```

```
erin      NaN
dave     68.0
carol    75.0
bob      82.0
anna     97.0
dtype: float64
```

```
pd.isna(s1)
```

```
erin      True
dave     False
carol    False
bob      False
anna     False
dtype: bool
```

```
pd.isna(s2)
```

```
erin      True
dave     False
carol    False
bob      False
anna     False
dtype: bool
```

Aside - why `pd.isna()`?

```
s = pd.Series([1,2,3,None]); s
```

```
0    1.0
1    2.0
2    3.0
3    NaN
dtype: float64
```

```
pd.isna(s)
```

```
0    False
1    False
2    False
3     True
dtype: bool
```

```
s == np.nan
```

```
0    False
1    False
2    False
3    False
dtype: bool
```

```
np.nan == np.nan
```

```
False
```

```
np.nan != np.nan
```

```
True
```

```
np.isnan(np.nan)
```

```
np.True_
```

```
np.isnan(0)
```

```
np.False_
```

Missing via None

In some cases None can also be used as a missing value, for example:

```
pd.Series([1,2,3,None])
```

```
0    1.0
1    2.0
2    3.0
3    NaN
dtype: float64
```

```
pd.Series([True,False,None])
```

```
0     True
1    False
2     None
dtype: object
```

```
pd.isna( pd.Series([1,2,3,None]) )
```

```
0    False
1    False
2    False
3     True
dtype: bool
```

```
pd.isna( pd.Series([True,False,None])
```

```
0    False
1    False
2     True
dtype: bool
```

This can have a side effect of changing the dtype of the series.

Native NAs

If instead of using base dtypes we use Pandas' built-in dtypes we get "native" support for missing values,

```
pd.Series(
  [1,2,3,None],
  dtype = pd.Int64Dtype()
)
```

```
0       1
1       2
2       3
3    <NA>
dtype: Int64
```

```
pd.Series(
  [True, False,None],
  dtype = pd.BooleanDtype()
)
```

```
0     True
1    False
2     <NA>
dtype: boolean
```

String series

Series containing strings can have their strings accessed via the `str` attribute,

```
s = pd.Series(["the quick", "brown fox", "jumps over", "a lazy dog"])
```

```
s
```

```
0     the quick
1     brown fox
2    jumps over
3    a lazy dog
dtype: str
```

```
s.str.split(" ")
```

```
0      [the, quick]
1      [brown, fox]
2     [jumps, over]
3    [a, lazy, dog]
dtype: object
```

```
s.str.upper()
```

```
0     THE QUICK
1     BROWN FOX
2    JUMPS OVER
3    A LAZY DOG
dtype: str
```

```
s.str.split(" ").str[1]
```

```
0    quick
1      fox
2     over
3     lazy
dtype: object
```

```
pd.Series([1,2,3]).str
```

```
AttributeError: Can only use .str accessor with string values, not integer. Did you mean: 'std'?
```

Categorical Series

```python
pd.Series(
  ["Mon", "Tue", "Wed", "Thu", "Fri"]
)
```

```
0    Mon
1    Tue
2    Wed
3    Thu
4    Fri
dtype: str
```

```python
pd.Series(
  ["Mon", "Tue", "Wed", "Thu", "Fri"],
  dtype="category"
)
```

```
0    Mon
1    Tue
2    Wed
3    Thu
4    Fri
dtype: category
Categories (5, str): ['Fri', 'Mon', 'Thu', 'Tue'
```

```python
pd.Series(
  ["Mon", "Tue", "Wed", "Thu", "Fri"],
  dtype=pd.CategoricalDtype(ordered=True)
)
```

```
0    Mon
1    Tue
2    Wed
3    Thu
4    Fri
dtype: category
Categories (5, str): ['Fri' < 'Mon' < 'Thu' < 'Tue' < 'Wed']
```

Category orders

```python
pd.Series(
  ["Tue", "Thu", "Mon", "Sat"],
  dtype=pd.CategoricalDtype(
    categories=["Mon", "Tue", "Wed", "Thu", "Fri"],
    ordered=True
  )
)
```

```
0    Tue
1    Thu
2    Mon
3    NaN
dtype: category
Categories (5, str): ['Mon' < 'Tue' < 'Wed' < 'Thu' < 'Fri']
```

DataFrames

DataFrame

- Just like R a DataFrame is a collection of vectors (Series) with a common length (and a common index)
- Column dtypes can be heterogeneous
- Columns have names stored in the `columns` index.
- It can be useful to think of a dictionary of Series objects where the keys are the column names.

```
iris = pd.read_csv("data/iris.csv")
type(iris)
```

```
<class 'pandas.DataFrame'>
```

```
iris
```

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

Constructing DataFrames

We just saw how to read a DataFrame via `read_csv()`, `DataFrames` can also be constructed via `DataFrame()`, in general this is done using a dictionary of columns / `Series`.

```
n = 5
d = {
  "id":     np.random.randint(100, 999, n),
  "weight": np.random.normal(70, 20, n),
  "height": np.random.normal(170, 15, n),
  "date":   pd.date_range(start='2/1/2026', periods=n, freq='D')
}
```

```
df = pd.DataFrame(d); df
```

	id	weight	height	date
0	659	74.851382	172.802779	2026-02-01
1	610	112.984848	187.245093	2026-02-02
2	339	65.630906	160.493238	2026-02-03
3	866	70.251167	185.992392	2026-02-04
4	626	70.038094	157.697184	2026-02-05

DataFrame from nparray

2d ndarrays can also be used to construct a `DataFrame` - generally it is a good idea to provide column and row names (indexes)

```
pd.DataFrame(
  np.diag([1,2,3]),
  columns = ["x","y","z"]
)
```

	x	y	z
0	1	0	0
1	0	2	0
2	0	0	3

```
pd.DataFrame(
  np.diag([1,2,3]),
  index = ["x","y","z"]
)
```

	0	1	2
x	1	0	0
y	0	2	0
z	0	0	3

```
pd.DataFrame(
  np.tri(5,3,-1),
  columns = ["x","y","z"],
  index = ["a","b","c","d","e"]
)
```

	x	y	z
a	0.0	0.0	0.0
b	1.0	0.0	0.0
c	1.0	1.0	0.0
d	1.0	1.0	1.0
e	1.0	1.0	1.0

DataFrame attributes & methods

```python
df.size
```

```
20
```

```python
df.shape
```

```
(5, 4)
```

```python
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   id      5 non-null      int64
 1   weight  5 non-null      float64
 2   height  5 non-null      float64
 3   date    5 non-null      datetime64[us]
dtypes: datetime64[us](1), float64(2), int64(1)
memory usage: 292.0 bytes
```

```python
df.dtypes
```

```
id                  int64
weight            float64
height            float64
date      datetime64[us]
dtype: object
```

```python
df.columns
```

```
Index(['id', 'weight', 'height', 'date'], dtype=
```

```python
df.index
```

```
RangeIndex(start=0, stop=5, step=1)
```

```python
df.axes
```

```
[RangeIndex(start=0, stop=5, step=1), Index(['id
```

DataFrame indexing

Selecting a column - use name or via the `.` accessor,

```
df["id"]
```

```
0    659
1    610
2    339
3    866
4    626
Name: id, dtype: int64
```

```
df.id
```

```
0    659
1    610
2    339
3    866
4    626
Name: id, dtype: int64
```

Selecting rows - use a single slice

```
df[1:3]
```

	id	weight	height	date
1	610	112.984848	187.245093	2026-02-02
2	339	65.630906	160.493238	2026-02-03

```
df[0::2]
```

	id	weight	height	date
0	659	74.851382	172.802779	2026-02-01
2	339	65.630906	160.493238	2026-02-03
4	626	70.038094	157.697184	2026-02-05

Integer based subsets are not allowed:

```
df[0]
```

```
KeyError: 0
```

Indexing by position

As with Series, position based indexing is done via the `iloc` attribute

```
df.iloc[1]
```

```
id                          610
weight               112.984848
height               187.245093
date       2026-02-02 00:00:00
Name: 1, dtype: object
```

```
df.iloc[[1]]
```

	id	weight	height	date
1	610	112.984848	187.245093	2026-02-02

```
df.iloc[0:2]
```

	id	weight	height	date
0	659	74.851382	172.802779	2026-02-01
1	610	112.984848	187.245093	2026-02-02

```
df.iloc[1:3,1:3]
```

	weight	height
1	112.984848	187.245093
2	65.630906	160.493238

```
df.iloc[0:3, [0,3]]
```

	id	date
0	659	2026-02-01
1	610	2026-02-02
2	339	2026-02-03

```
df.iloc[0:3, [True, True, False, False]]
```

	id	weight
0	659	74.851382
1	610	112.984848
2	339	65.630906

```
df.iloc[lambda x: x.index % 2 != 0]
```

	id	weight	height	date
1	610	112.984848	187.245093	2026-02-02
3	866	70.251167	185.992392	2026-02-04

Index by name

```
df.index = (["anna","bob","carol", "dave", "erin"]); df
```

	id	weight	height	date
anna	659	74.851382	172.802779	2026-02-01
bob	610	112.984848	187.245093	2026-02-02
carol	339	65.630906	160.493238	2026-02-03
dave	866	70.251167	185.992392	2026-02-04
erin	626	70.038094	157.697184	2026-02-05

```
df.loc["anna"]
```

```
id                          659
weight                74.851382
height               172.802779
date       2026-02-01 00:00:00
Name: anna, dtype: object
```

```
df.loc[["anna"]]
```

	id	weight	height	date
anna	659	74.851382	172.802779	2026-02-01

```
type(df.loc["anna"])
```

```
<class 'pandas.Series'>
```

```
type(df.loc[["anna"]])
```

```
<class 'pandas.DataFrame'>
```

```python
df.loc["bob":"dave"]
```

	id	weight	height	date
bob	610	112.984848	187.245093	2026-02-02
carol	339	65.630906	160.493238	2026-02-03
dave	866	70.251167	185.992392	2026-02-04

```python
df.loc[df.id < 300]
```

id	weight	height	date

```python
df.loc[:, "date"]
```

```
anna    2026-02-01
bob     2026-02-02
carol   2026-02-03
dave    2026-02-04
erin    2026-02-05
Name: date, dtype: datetime64[us]
```

```python
df.loc[["bob","erin"], "weight":"height"]
```

	weight	height
bob	112.984848	187.245093
erin	70.038094	157.697184

```python
df.loc[0:2, "weight":"height"]
```

```
TypeError: cannot do slice indexing on Index with these indexers [0] of type int
```

Views vs. Copies

In general most pandas operations will generate a new object. Previously some subset operations would return views, this has mostly been resolved with pandas 3.0.0.

```
d = pd.DataFrame(np.arange(6).reshape(3,2),
```

	x	y
0	0	1
1	2	3
2	4	5

```
v = d.iloc[0:2,0:2]; v
```

	x	y
0	0	1
1	2	3

```
d.iloc[0,1] = -1; d
```

	x	y
0	0	-1
1	2	3
2	4	5

```
v
```

	x	y
0	0	1
1	2	3

```
v.iloc[0,0] = 100; v
```

	x	y
0	100	1
1	2	3

```
d
```

	x	y
0	0	-1
1	2	3
2	4	5

Element access

```
df
```

	id	weight	height	date
anna	659	74.851382	172.802779	2026-02-01
bob	610	112.984848	187.245093	2026-02-02
carol	339	65.630906	160.493238	2026-02-03
dave	866	70.251167	185.992392	2026-02-04
erin	626	70.038094	157.697184	2026-02-05

```
df[0,0]
```

```
KeyError: (0, 0)
```

```
df.iat[0,0]
```

```
np.int64(659)
```

```
df.id[0]
```

```
KeyError: 0
```

```
df[0:1].id[0]
```

```
KeyError: 0
```

```
df["anna", "id"]
```

```
KeyError: ('anna', 'id')
```

```
df.at["anna", "id"]
```

```
np.int64(659)
```

```
df["id"]["anna"]
```

```
np.int64(659)
```

```
df["id"][0]
```

```
KeyError: 0
```

Index objects

Columns and index

When constructing a DataFrame we can specify the indexes for both the rows (`index`) and columns (`columns`),

```
df1 = pd.DataFrame(
  np.random.randn(5, 3),
  columns=['A', 'B', 'C']
)
df1
```

	A	B	C
0	0.142559	-2.549740	-0.827619
1	0.090425	1.525071	-0.707434
2	0.415434	0.643247	-0.323415
3	2.594474	-0.957565	-1.409895
4	0.042587	1.650910	-0.474850

```
df2 = pd.DataFrame(
  np.random.randn(3, 3),
  index=['x','y','z'],
  columns=['A', 'B', 'C']
)
df2
```

	A	B	C
x	1.153564	0.607246	-0.745101
y	-0.440562	1.118582	-0.927971
z	-0.440077	-1.004909	-1.058644

```
df1.columns
```

```
Index(['A', 'B', 'C'], dtype='str')
```

```
df1.index
```

```
RangeIndex(start=0, stop=5, step=1)
```

```
df2.columns
```

```
Index(['A', 'B', 'C'], dtype='str')
```

```
df2.index
```

```
Index(['x', 'y', 'z'], dtype='str')
```

Index objects

pandas' `Index` class and its subclasses provide the infrastructure necessary for lookups, data alignment, and other related tasks. You can think of them as being an immutable *multiset* (i.e. duplicate values are allowed).

```
pd.Index(['A','B','C'])
```

```
Index(['A', 'B', 'C'], dtype='str')
```

```
pd.Index(['A','B','C','A'])
```

```
Index(['A', 'B', 'C', 'A'], dtype='str')
```

```
pd.Index(range(5))
```

```
RangeIndex(start=0, stop=5, step=1)
```

```
pd.Index(list(range(5)))
```

```
Index([0, 1, 2, 3, 4], dtype='int64')
```

Index names

Index objects can have names which are shown when printing the DataFrame or Index,

```
df = pd.DataFrame(
  np.random.randn(3, 3),
  index=pd.Index(['x','y','z'], name="rows"),
  columns=pd.Index(['A', 'B', 'C'], name="cols")
)
df
```

cols	A	B	C
rows			
x	-2.004805	-0.317074	0.319219
y	-0.259864	0.725242	-0.450637
z	0.409833	-0.450260	0.818892

```
df.columns
```

```
Index(['A', 'B', 'C'], dtype='str', name='cols')
```

```
df.index
```

```
Index(['x', 'y', 'z'], dtype='str', name='rows')
```

Indexes and missing values

It is possible for an index to contain missing values (e.g. `np.nan`) but this is generally a bad idea and should be avoided.

```
pd.Index([1,2,3,np.nan,5])
```

```
Index([1.0, 2.0, 3.0, nan, 5.0], dtype='float64')
```

```
pd.Index(["A","B",np.nan,"D", None])
```

```
Index(['A', 'B', nan, 'D', nan], dtype='str')
```

Missing values can be replaced via the `fillna()` method,

```
pd.Index([1,2,3,np.nan,5]).fillna(0)
```

```
Index([1.0, 2.0, 3.0, 0.0, 5.0], dtype='float64')
```

```
pd.Index(["A","B",np.nan,"D", None]).fillna("Z")
```

```
Index(['A', 'B', 'Z', 'D', 'Z'], dtype='str')
```

Changing a DataFrame's index

Existing columns can be made into an index via `set_index()` and removed via `reset_index()`,

```
data
```

	a	b	c	d
0	bar	one	z	1
1	bar	two	y	2
2	foo	one	x	3
3	foo	two	w	4

```
data.set_index('a')
```

	b	c	d
a			
bar	one	z	1
bar	two	y	2
foo	one	x	3
foo	two	w	4

```
data.set_index('a').reset_index()
```

	a	b	c	d
0	bar	one	z	1
1	bar	two	y	2
2	foo	one	x	3
3	foo	two	w	4

drop argument

```
data.set_index('c', drop=False)
```

	a	b	c	d
c				
z	bar	one	z	1
y	bar	two	y	2
x	foo	one	x	3
w	foo	two	w	4

```
data.set_index('c').reset_index(drop=True)
```

	a	b	d
0	bar	one	1
1	bar	two	2
2	foo	one	3
3	foo	two	4

Creating a new index

New index values can be attached to a DataFrame via `reindex()`,

```
data
```

	a	b	c	d
0	bar	one	z	1
1	bar	two	y	2
2	foo	one	x	3
3	foo	two	w	4

```
data.reindex(columns=["a","b","c","d","e"])
```

	a	b	c	d	e
0	bar	one	z	1	NaN
1	bar	two	y	2	NaN
2	foo	one	x	3	NaN
3	foo	two	w	4	NaN

```
data.reindex(["w","x","y","z"])
```

	a	b	c	d
w	NaN	NaN	NaN	NaN
x	NaN	NaN	NaN	NaN
y	NaN	NaN	NaN	NaN
z	NaN	NaN	NaN	NaN

```
data.index = ["w","x","y","z"]; data
```

	a	b	c	d
w	bar	one	z	1
x	bar	two	y	2
y	foo	one	x	3
z	foo	two	w	4

```
data.reindex(range(4,0,-1))
```

	a	b	c	d
4	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN

```
data.index = range(4,0,-1); data
```

	a	b	c	d
4	bar	one	z	1
3	bar	two	y	2
2	foo	one	x	3
1	foo	two	w	4

MultiIndexes

MultiIndex objects

These are a hierarchical analog of standard Index objects and are used to represent nested indexes. There are a number of methods for constructing them based on the initial object

```
tuples = [('A','x'), ('A','y'),
          ('B','x'), ('B','y'),
          ('C','x'), ('C','y')]
pd.MultiIndex.from_tuples(
  tuples, names=["1st","2nd"]
)
```

```
MultiIndex([('A', 'x'),
            ('A', 'y'),
            ('B', 'x'),
            ('B', 'y'),
            ('C', 'x'),
            ('C', 'y')],
           names=['1st', '2nd'])
```

```
pd.MultiIndex.from_product(
  [["A","B","C"],
   ["x","y"]],
  names=["1st","2nd"]
)
```

```
MultiIndex([('A', 'x'),
            ('A', 'y'),
            ('B', 'x'),
            ('B', 'y'),
            ('C', 'x'),
            ('C', 'y')],
           names=['1st', '2nd'])
```

DataFrame with MultiIndex

```
idx = pd.MultiIndex.from_tuples(
  tuples, names=["1st","2nd"]
)

pd.DataFrame(
  np.random.rand(6,2),
  index = idx,
  columns=["m","n"]
)
```

		m	n
1st	**2nd**		
A	**x**	0.762480	0.597482
	y	0.518560	0.208513
B	**x**	0.871525	0.974238
	y	0.846505	0.649222
C	**x**	0.458415	0.707400
	y	0.474986	0.144077

Column MultiIndex

MultiIndexes can also be used for columns as well,

```
cidx = pd.MultiIndex.from_product(
  [["A","B"],["x","y"]], names=["c1","c2"]
)

pd.DataFrame(
  np.random.rand(4,4), columns = cidx
)
```

c1	A		B	
c2	x	y	x	y
0	0.963868	0.556983	0.971970	0.437350
1	0.816012	0.322633	0.403163	0.810778
2	0.468359	0.732775	0.250353	0.043750
3	0.357053	0.125794	0.510254	0.481961

```
ridx = pd.MultiIndex.from_product(
  [["m","n"],["l","p"]], names=["r1","r2"]
)

pd.DataFrame(
  np.random.rand(4,4),
  index= ridx, columns = cidx
)
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	l	0.795220	0.009661	0.843386	0.046197
	p	0.459810	0.004418	0.725141	0.385983
n	l	0.627062	0.263611	0.427911	0.617691
	p	0.746212	0.229924	0.831836	0.737049

MultiIndex indexing

```
data
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	l	0.922092	0.562895	0.229422	0.810382
	p	0.629027	0.054008	0.481722	0.899993
n	l	0.551053	0.651368	0.202971	0.128696
	p	0.000461	0.017242	0.146753	0.268246

```
data["A"]
```

	c2	x	y
r1	r2		
m	l	0.922092	0.562895
	p	0.629027	0.054008
n	l	0.551053	0.651368
	p	0.000461	0.017242

```
data["x"]
```

```
KeyError: 'x'
```

```
data["m"]
```

```
KeyError: 'm'
```

```
data["m","A"]
```

```
KeyError: ('m', 'A')
```

```
data["A","x"]
```

```
r1  r2
m   l     0.922092
    p     0.629027
n   l     0.551053
    p     0.000461
Name: (A, x), dtype: float64
```

```
data["A"]["x"]
```

```
r1  r2
m   l     0.922092
    p     0.629027
n   l     0.551053
    p     0.000461
Name: x, dtype: float64
```

MultiIndex indexing via `iloc`

```
data.iloc[0]
```

```
c1  c2
A   x     0.922092
    y     0.562895
B   x     0.229422
    y     0.810382
Name: (m, l), dtype: float64
```

```
type(data.iloc[0])
```

```
<class 'pandas.Series'>
```

```
data.iloc[(0,1)]
```

```
np.float64(0.5628949648304943)
```

```
data.iloc[[0,1]]
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	l	0.922092	0.562895	0.229422	0.810382
	p	0.629027	0.054008	0.481722	0.899993

```
data.iloc[:,0]
```

```
r1  r2
m   l     0.922092
    p     0.629027
n   l     0.551053
    p     0.000461
Name: (A, x), dtype: float64
```

```
type(data.iloc[:,0])
```

```
<class 'pandas.Series'>
```

```
data.iloc[0,1]
```

```
np.float64(0.5628949648304943)
```

```
data.iloc[0,[0,1]]
```

```
c1  c2
A   x     0.922092
    y     0.562895
Name: (m, l), dtype: float64
```

MultiIndex indexing via `loc`

```
data.loc["m"]
```

c1	A		B	
c2	x	y	x	y
r2				
l	0.922092	0.562895	0.229422	0.810382
p	0.629027	0.054008	0.481722	0.899993

```
data.loc["l"]
```

```
KeyError: 'l'
```

```
data.loc[:,"A"]
```

	c2	x	y
r1	r2		
m	l	0.922092	0.562895
	p	0.629027	0.054008
n	l	0.551053	0.651368
	p	0.000461	0.017242

```
data.loc[("m","l")]
```

```
c1  c2
A   x     0.922092
    y     0.562895
B   x     0.229422
    y     0.810382
Name: (m, l), dtype: float64
```

```
data.loc[:,("A","y")]
```

```
r1  r2
m   l     0.562895
    p     0.054008
n   l     0.651368
    p     0.017242
Name: (A, y), dtype: float64
```

Fancier indexing with `loc`

Index slices can also be used with combinations of indexes and index tuples,

```
data.loc["m":"n"]
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	l	0.922092	0.562895	0.229422	0.810382
	p	0.629027	0.054008	0.481722	0.899993
n	l	0.551053	0.651368	0.202971	0.128696
	p	0.000461	0.017242	0.146753	0.268246

```
data.loc[("m","p"):"n"]
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	p	0.629027	0.054008	0.481722	0.899993
n	l	0.551053	0.651368	0.202971	0.128696
	p	0.000461	0.017242	0.146753	0.268246

```
data.loc[("m","l"):("n","l")]
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	l	0.922092	0.562895	0.229422	0.810382
	p	0.629027	0.054008	0.481722	0.899993
n	l	0.551053	0.651368	0.202971	0.128696

```
data.loc[[("m","p"),("n","l")]]
```

	c1	A		B	
	c2	x	y	x	y
r1	r2				
m	p	0.629027	0.054008	0.481722	0.899993
n	l	0.551053	0.651368	0.202971	0.128696

Selecting nested levels

The previous methods don't give easy access to indexing on nested index levels, this is possible via the cross-section method `xs()`,

```
data.xs("p", level="r2")
```

c1	A		B	
c2	x	y	x	y
r1				
m	0.629027	0.054008	0.481722	0.899993
n	0.000461	0.017242	0.146753	0.268246

```
data.xs("y", level="c2", axis=1)
```

	c1	A	B
r1	r2		
m	l	0.562895	0.810382
	p	0.054008	0.899993
n	l	0.651368	0.128696
	p	0.017242	0.268246

```
data.xs("m", level="r1")
```

c1	A		B	
c2	x	y	x	y
r2				
l	0.922092	0.562895	0.229422	0.810382
p	0.629027	0.054008	0.481722	0.899993

```
data.xs("B", level="c1", axis=1)
```

	c2	x	y
r1	r2		
m	l	0.229422	0.810382
	p	0.481722	0.899993
n	l	0.202971	0.128696
	p	0.146753	0.268246

Setting MultiIndexes

It is also possible to construct a MultiIndex or modify an existing one using `set_index()` and `reset_index()`,

```
data
```

	a	b	c	d
0	bar	one	z	1
1	bar	two	y	2
2	foo	one	x	3

```
data.set_index(['a','b'])
```

		c	d
a	**b**		
bar	**one**	z	1
	two	y	2
foo	**one**	x	3

```
data.set_index(['a','b']).reset_index()
```

	a	b	c	d
0	bar	one	z	1
1	bar	two	y	2
2	foo	one	x	3

```
data.set_index('c', append=True)
```

		a	b	d
	c			
0	**z**	bar	one	1
1	**y**	bar	two	2
2	**x**	foo	one	3

```
data.set_index(['a','b']).reset_index(level=1)
```

	b	c	d
a			
bar	one	z	1
bar	two	y	2
foo	one	x	3

Working with DataFrames

Filtering rows

The `query()` method can be used for filtering rows, it evaluates a string expression in the context of the data frame.

```
df.query('date == "2026-02-01"')
```

	id	weight	height	date

```
df.query('weight > 50')
```

	id	weight	height	date
anna	202	82.953771	193.688192	2026-02-01
bob	535	100.460597	181.511521	2026-02-02
carol	960	65.316933	162.957884	2026-02-03
dave	370	65.317261	178.138401	2026-02-04

```
df.query('weight > 50 & height < 165')
```

	id	weight	height	date
carol	960	65.316933	162.957884	2026-02-03

```
qid = 202
df.query('id == @qid')
```

	id	weight	height	date
anna	202	82.953771	193.688192	2026-02-01

Selecting Columns

Beyond the use of `loc()` and `iloc()` there is also the `filter()` method which can be used to select columns (or indices) by name with pattern matching

```
df.filter(items=["id","weight"])
```

	id	weight
anna	202	82.953771
bob	535	100.460597
carol	960	65.316933
dave	370	65.317261

```
df.filter(regex="ght$")
```

	weight	height
anna	82.953771	193.688192
bob	100.460597	181.511521
carol	65.316933	162.957884
dave	65.317261	178.138401

```
df.filter(like = "i")
```

	id	weight	height
anna	202	82.953771	193.688192
bob	535	100.460597	181.511521
carol	960	65.316933	162.957884
dave	370	65.317261	178.138401

```
df.filter(like="a", axis=0)
```

	id	weight	height	date
anna	202	82.953771	193.688192	2026-02-01
carol	960	65.316933	162.957884	2026-02-03
dave	370	65.317261	178.138401	2026-02-04

Adding columns

Indexing with assignment allows for inplace modification of a DataFrame, while `assign()` creates a new object (but is chainable)

```
df['student'] = [True, True, True, None]
df['age'] = [19, 22, None, None]
df
```

	id	weight	height	date	student	age
anna	202	82.953771	193.688192	2026-02-01	True	19.0
bob	535	100.460597	181.511521	2026-02-02	True	22.0
carol	960	65.316933	162.957884	2026-02-03	True	NaN
dave	370	65.317261	178.138401	2026-02-04	None	NaN

```
df.assign(
  student = lambda x: np.where(x.student, "yes", "no"),
  rand = np.random.rand(n)
)
```

	id	weight	height	date	student	age	rand
anna	202	82.953771	193.688192	2026-02-01	yes	19.0	0.181825
bob	535	100.460597	181.511521	2026-02-02	yes	22.0	0.183405
carol	960	65.316933	162.957884	2026-02-03	yes	NaN	0.304242
dave	370	65.317261	178.138401	2026-02-04	no	NaN	0.524756

pd.col()

As of pandas 3.0.0 there is also a `pd.col()` function which can be used within methods like `assign()` and `query()` to refer to columns by name,

```
df.assign(
  bmi = pd.col("weight") / np.power(pd.col("height")/100, 2)
)
```

	id	weight	height	date	student	age	bmi
anna	202	82.953771	193.688192	2026-02-01	True	19.0	22.112092
bob	535	100.460597	181.511521	2026-02-02	True	22.0	30.492102
carol	960	65.316933	162.957884	2026-02-03	True	NaN	24.596597
dave	370	65.317261	178.138401	2026-02-04	None	NaN	20.583199

It is still early days for `pd.col()` so expect some rough edges - e.g. `(pd.col("height")/100) ** 2` did not work as

Removing columns (and rows)

Columns or rows can be removed via the `drop()` method,

```
df.drop(['student'])
```

```
KeyError: "['student'] not found in axis"
```

```
df.drop(['student'], axis=1)
```

	id	weight	height	date	age
anna	202	82.953771	193.688192	2026-02-01	19.0
bob	535	100.460597	181.511521	2026-02-02	22.0
carol	960	65.316933	162.957884	2026-02-03	NaN
dave	370	65.317261	178.138401	2026-02-04	NaN

```
df.drop(['anna','dave'])
```

	id	weight	height	date	student	age
bob	535	100.460597	181.511521	2026-02-02	True	22.0
carol	960	65.316933	162.957884	2026-02-03	True	NaN

```python
df.drop(columns = df.columns == "age")
```

```
KeyError: '[False, False, False, False, False, True] not found in axis'
```

```python
df.drop(columns = df.columns[df.columns == "age"])
```

	id	weight	height	date	student
anna	202	82.953771	193.688192	2026-02-01	True
bob	535	100.460597	181.511521	2026-02-02	True
carol	960	65.316933	162.957884	2026-02-03	True
dave	370	65.317261	178.138401	2026-02-04	None

```python
df.drop(columns = df.columns[df.columns.str.contains("ght")])
```

	id	date	student	age
anna	202	2026-02-01	True	19.0
bob	535	2026-02-02	True	22.0
carol	960	2026-02-03	True	NaN
dave	370	2026-02-04	None	NaN

Sorting

DataFrames can be sorted on one or more columns via `sort_values()`,

```
df
```

	id	weight	height	date	student	age
anna	202	82.953771	193.688192	2026-02-01	True	19.0
bob	535	100.460597	181.511521	2026-02-02	True	22.0
carol	960	65.316933	162.957884	2026-02-03	True	NaN
dave	370	65.317261	178.138401	2026-02-04	None	NaN

```
df.sort_values(by=["student","id"], ascending=[True,False])
```

	id	weight	height	date	student	age
carol	960	65.316933	162.957884	2026-02-03	True	NaN
bob	535	100.460597	181.511521	2026-02-02	True	22.0
anna	202	82.953771	193.688192	2026-02-01	True	19.0
dave	370	65.317261	178.138401	2026-02-04	None	NaN

join vs merge vs concat

All three can be used to combine data frames,

- `concat()` stacks DataFrames on either axis, with basic alignment based on (row) indexes. `join` argument only supports “inner” and “outer”.
- `merge()` aligns based on one or more shared columns. `how` supports “inner”, “outer”, “left”, “right”, and “cross”.
- `join()` uses `merge()` behind the scenes, but prefers to join based on (row) indexes. Also has different default `how` compared to `merge()`, “left” vs “inner”.

Next week

- Pivoting
- Split-Apply-Combine (group by / summarize)
- Pandas & pyarrow
- Polars